

Características bases del lenguaje javascript(interpretado, dinámico, multiparadigma,etc)

Las características base que definen su naturaleza son:

- **Interpretado (y compilado Just-In-Time):** No necesita un proceso de compilación previo antes de ejecutarse. Un motor en el navegador o entorno de ejecución lee y traduce el código directamente línea por línea. Hoy en día, los motores usan compilación *Just-In-Time* (JIT) para mejorar el rendimiento.
- **Multiparadigma:** Soporta diversos estilos de programación. Se puede programar de manera **imperativa/estructurada** (usando bucles y condiciones), **orientada a objetos** (a través de prototipos o clases) y **funcional** (tratando las funciones como ciudadanos de primera clase).
- **Dinámico:** Realiza la comprobación de tipos y la vinculación de objetos en tiempo de ejecución. Esto significa que las variables no están atadas a un tipo de dato específico y pueden cambiar su estructura sobre la marcha.
- **Tipado débil (o dinámico):** No requiere declarar explícitamente el tipo de dato de una variable al crearla. El lenguaje realiza conversiones automáticas e implícitas al realizar operaciones (por ejemplo, sumar un número a un texto).
- **Orientado a objetos basado en prototipos:** A diferencia de lenguajes como Java o C++, JavaScript no utiliza clases tradicionales (hasta la introducción de `class` como azúcar sintáctico en ES6), sino que los objetos heredan propiedades directamente de otros objetos (prototipos).
- **Asíncrono y dirigido por eventos:** Utiliza un modelo de concurrencia basado en un **bucle de eventos** (*Event Loop*). Esto le permite ejecutar operaciones largas o de red en segundo plano (asincronismo) sin bloquear el hilo principal, utilizando promesas (Promises) o `async/await`.

Estructuras Condicionales

- **if / else if / else:** Ejecuta bloques de código basados en evaluaciones booleanas verdaderas o falsas.
- **switch:** Evalúa una expresión comparando su valor con múltiples casos (case) usando igualdad estricta (`===`).
- **Operador Ternario (`? :`):** Alternativa compacta al `if / else` para asignar valores en una sola línea.

Bucles e Iteraciones

- **for**: Repite un bloque de código un número específico de veces controlando un contador inicial, una condición y un incremento.
- **while**: Ejecuta el código continuamente mientras la condición evaluada sea verdadera.
- **do ... while**: Garantiza que el código se ejecute al menos una vez antes de evaluar la condición.
- **for ... of**: Recorre los valores de objetos iterables como arreglos (Arrays), cadenas de texto (Strings) o colecciones (Maps/Sets).
- **for ... in**: Recorre las propiedades enumerables (claves/índices) de un objeto o arreglo.

Control de Bucles

- **break**: Termina de forma inmediata la ejecución del bucle actual.
- **continue**: Salta el resto de la iteración actual y pasa directamente a la siguiente.

Tipos de Datos

Primitivos

Se almacenan directamente en la pila de memoria (*stack*) por su valor. Son **inmutables** (no se pueden modificar, solo reemplazar) y se comparan por su **valor**.

- **Number**: Números enteros y decimales (ej. 42, 3.14). Incluye Infinity y NaN.
- **String**: Cadenas de texto delimitadas por comillas simples, dobles o *backticks* (ej. "Hola").
- **Boolean**: Solo admite dos valores lógicos: true o false.
- **Undefined**: Valor asignado automáticamente a variables declaradas que aún no tienen valor.
- **Null**: Representa intencionalmente la ausencia de cualquier valor de objeto.
- **BigInt**: Para números enteros con precisión arbitraria más grandes que el límite seguro de Number.
- **Symbol**: Identificadores únicos y anónimos, usados frecuentemente como claves de propiedades de objetos.

Referencia (Objetos)

No guardan el valor directamente en la variable, sino una dirección o **referencia** apuntando a un espacio en la memoria dinámica (*heap*). Son **mutables** y se comparan por su **referencia** (dirección de memoria), no por su contenido.

- **Object**: Estructuras de pares clave-valor (ej. { nombre: "Ana", edad: 25 }).
- **Array**: Listas ordenadas de datos indexadas numéricamente (ej. [1, 2, 3]).
- **Function**: Bloques de código ejecutables que en JavaScript se tratan como objetos.

- **Colecciones especiales:** Estructuras avanzadas de almacenamiento como Map, Set, Date y RegExp.

Funciones

1. Funciones Declaradas (Function Declarations)

Se definen utilizando la palabra clave function seguida obligatoriamente por el nombre de la función.

- **Sintaxis:** function sumar(a, b) { return a + b; }
- **Hoisting:** Tienen *hoisting* completo. El motor de JavaScript las eleva al inicio de su contexto de ejecución, por lo que **pueden ser llamadas antes de su línea de definición**.
- **Contexto:** Tienen su propio objeto this según cómo sean invocadas.

2. Funciones Expresadas (Function Expressions)

Se definen creando una función (generalmente anónima) y asignándola a una variable (const o let).

- **Sintaxis:** const restar = function(a, b) { return a - b; };
- **Hoisting:** No sufren *hoisting* de función. La variable es elevada, pero su valor es undefined hasta que el flujo llega a la línea de asignación. **No se pueden llamar antes de ser definidas**.
- **Contexto:** Al igual que las declaradas, su this se determina en tiempo de ejecución.

3. Funciones Flecha (Arrow Functions)

Introducidas en ECMAScript 6, ofrecen una sintaxis más corta y compacta. Se omiten las palabras clave function y return (si es de una sola línea).

- **Sintaxis:** const multiplicar = (a, b) => a * b;
- **Retorno implícito:** Si no lleva llaves { }, retorna el resultado de la expresión automáticamente.
- **Hoisting:** Al ser asignadas a variables, siguen las mismas reglas de restricción que las funciones expresadas.
- **Contexto this léxico:** No crean su propio contexto de this. Heredan el this del entorno que las rodea en el momento de su definición, lo que las hace ideales para métodos dentro de callbacks o temporizadores.
- **Limitaciones:** No tienen acceso al objeto arguments y **no pueden ser usadas como constructores** (con el operador new).

Objetos y Arrays

Objetos (Objects)

Son colecciones de propiedades compuestas por pares de **clave-valor**. Las claves son cadenas de texto o símbolos, y los valores pueden ser cualquier tipo de dato (incluyendo funciones, llamadas entonces métodos).

- **Creación:** Se utiliza la sintaxis de llaves {} (objeto literal).
- **Acceso:** Se accede a sus valores mediante la notación de punto (objeto.propiedad) o de corchetes (objeto["propiedad"]).
- **Dinámicos:** Se pueden agregar, modificar o eliminar propiedades en cualquier momento de la ejecución.

Arreglos (Arrays)

Son colecciones ordenadas de elementos indexados numéricamente, donde el primer elemento siempre empieza en el índice 0.

- **Creación:** Se utiliza la sintaxis de corchetes [] (arreglo literal).
- **Heterogéneos:** Pueden almacenar múltiples tipos de datos simultáneamente en la misma lista.
- **Propiedad length:** Refleja automáticamente la cantidad total de elementos dentro del arreglo.
- **Métodos integrados:** Incluyen funciones nativas para manipular datos, tales como mutables (push, pop, shift, unshift) e inmutables/funcionales (map, filter, reduce).

Manipulación del DOM

El DOM representa el documento HTML como un árbol de nodos que JavaScript puede modificar sobre la marcha.

- **Selección de elementos:**
 - document.getElementById('id'): Selecciona un elemento único por su ID.
 - document.querySelector('.clase'): Selecciona el primer elemento que coincida con el selector CSS.
 - document.querySelectorAll('p'): Selecciona todos los elementos que coincidan y devuelve una lista estática (NodeList).
- **Modificación de contenido:**
 - elemento.textContent: Modifica o lee el texto plano dentro del elemento (seguro contra XSS).
 - elemento.innerHTML: Modifica o lee el contenido HTML interno.
- **Modificación de atributos y estilos:**
 - elemento.setAttribute('src', 'imagen.png'): Cambia atributos HTML.
 - elemento.classList.add('activa') / .remove() / .toggle() / .contains(): Gestiona clases CSS de forma eficiente.
 - elemento.style.color = 'red': Aplica estilos CSS en línea directamente.
- **Creación y eliminación de nodos:**
 - document.createElement('div'): Crea un nuevo elemento en memoria.
 - padre.appendChild(hijo): Inserta el elemento creado al final del contenedor padre.
 - elemento.remove(): Elimina el elemento directamente del DOM.

Manejo de Eventos

Un evento es una acción que ocurre en el navegador (un clic, pulsar una tecla, cargar la página) ante la cual podemos ejecutar código.

- **Escuchador de eventos (addEventListener):** Es el método estándar para registrar eventos sin pisar otros scripts.
 - **Eventos comunes:**
- **Mouse:** click, dblclick, mouseenter, mouseleave.
- **Teclado:** keydown, keyup.
- **Formulario:** submit (envío), change (cambio de valor), input (escritura en tiempo real).
- **Documento:** DOMContentLoaded (el HTML se ha cargado por completo).
 - **Control del flujo del evento:**
- evento.preventDefault(): Detiene la acción por defecto del navegador (como evitar que un formulario recargue la página).
- evento.stopPropagation(): Detiene la propagación del evento a través de los elementos padres (fase de burbujeo o *bubbling*).

Asincronía

1. Callbacks (Funciones de Retorno)

Es el mecanismo más antiguo. Consiste en pasar una función como argumento a otra función, para que se ejecute una vez que la tarea asíncrona haya terminado.

- **Problema Principal:** Provoca el **Callback Hell** (o pirámide de la perdición) cuando se encadenan múltiples operaciones, volviendo el código ilegible y difícil de mantener.

2. Promesas (Promises)

Introducidas en ES6 (2015) para solucionar el Callback Hell. Una promesa es un objeto que representa el éxito o el fracaso futuro de una operación asíncrona.

- **Estados:** Tiene tres estados posibles: pending (pendiente), fulfilled (resuelta con éxito) o rejected (rechazada con error).
- **Consumo:** Se gestionan mediante los métodos **.then()** (para el éxito) y **.catch()** (para capturar errores), permitiendo encadenar tareas linealmente.

3. Async / Await

Introducido en ES8 (2017), es **azúcar sintáctico** sobre las promesas. No cambia el comportamiento interno de JavaScript, pero permite escribir código asíncrono que se lee y se comporta de manera idéntica al código síncrono.

- **async:** Se coloca antes de la definición de una función para indicar que siempre retornará una promesa de forma implícita.
- **await:** Se usa dentro de funciones async para pausar la ejecución del bloque de código de esa función hasta que la promesa se resuelva o se rechace.
- **Manejo de Errores:** Se utiliza la estructura nativa **try ... catch** para gestionar los fallos de manera limpia.

JAVASCRIPT

Blanco Lizárraga Karol Sebastián

Características bases del lenguaje

Diseñado principalmente para dotar de interactividad a las páginas web. Es ejecutado de forma nativa por los navegadores web y, gracias a entornos como Node.js, también en el lado del servidor.

Estructuras de control

Las estructuras de control de JavaScript permiten modificar el flujo de ejecución del código según condiciones o repeticiones

Tipos de datos

En JavaScript, los tipos de datos se dividen estrictamente en dos categorías según cómo se almacenan y se manipulan en la memoria: Primitivos y de Referencia

Funciones

Las funciones en JavaScript son bloques de código reutilizables y se consideran "ciudadanos de primera clase", lo que significa que pueden asignarse a variables, pasarse como argumentos y ser retornadas por otras funciones.

Objetos y Arrays

En JavaScript, los objetos y los arreglos (arrays) son estructuras de datos de referencia fundamentales para agrupar y organizar información

Manipulación del DOM

La manipulación del DOM (Document Object Model) y el manejo de eventos permiten que JavaScript interactúe con el HTML para crear interfaces dinámicas

Manejo de eventos

Un evento es una acción que ocurre en el navegador (un clic, pulsar una tecla, cargar la página) ante la cual podemos ejecutar código

Asincronía

JavaScript ejecuta su código en un solo hilo (single-threaded). Para realizar tareas pesadas (como peticiones HTTP o temporizadores) sin congelar la interfaz web, utiliza un modelo de asincronismo respaldado por el Event Loop.

Características

Interpretado (JIT), Multiparadigma, Dinámico, Tipado débil, Orientado a objetos basado en prototipos, Asíncrono y dirigido por eventos.

Condiciones

if/else if/else, switch, operador ternario.

Bucles e Iteraciones

for, while, do...while, for...of, for...in.

Control de Bucles

- Continue
- Break

Primitivos

Number, String, Boolean, Undefined, Null, BigInt, Symbol.

Referencia

Object, Array, Function, Map, Set, Date, RegExp.

Funciones

Declaradas, Expresadas, Flecha (Arrow Functions).

Objetos

pares clave-valor, acceso mediante punto y corchetes, propiedades dinámicas.

Arrays

colecciones ordenadas, propiedad length, métodos push(), pop(), shift(), unshift(), map(), filter(), reduce().

Manipulación del DOM

pares clave-valor, acceso mediante punto y corchetes, propiedades dinámicas.

Manejo de eventos

addEventListener(), eventos de mouse, eventos de teclado, eventos de formularios, DOMContentLoaded, preventDefault(), stopPropagation().

Asincronía

Callbacks, Promesas (pending, fulfilled, rejected, then(), catch()), Async/Await (async, await, try...catch).

Ejemplo: Multiparadigma

```
const duplicados = numeros.map(num => num * 2); // [2, 4, 6]
```

Ejemplo: Callbacks

```
hacerPeticion1(() => { hacerPeticion2(() => { hacerPeticion3(() => { // Código anidado complejo }); }); });
```

Ejemplo: Primitivos

42, 3.14

Ejemplo: Objeto

·Se accede a sus valores mediante la notación de punto (objeto.propiedad) o de corchetes (objeto["propiedad"]).